Question 1: Hospital Appointment Management — SQL Joins

Create tables:

```
mysql> use dbms
Database changed
mysql> CREATE TABLE Patient (
    ->     Patient_ID INT PRIMARY KEY,
    ->     Patient_Name VARCHAR(100),
    ->     Age INT,
    ->     City VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.159 sec)

mysql>
mysql> CREATE TABLE Doctor (
    ->     Doctor_ID INT PRIMARY KEY,
    ->     Doctor_Name VARCHAR(100),
    ->     Specialization VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.247 sec)

mysql>
mysql> CREATE TABLE Appointment (
    ->     Appointment_ID INT PRIMARY KEY,
    ->     Patient_ID INT,
    ->     Doctor_ID INT,
    ->     Appointment_Date DATE,
    ->     Status VARCHAR(50),
    ->     FOREIGN KEY (Patient_ID) REFERENCES Patient(Patient_ID),
    ->     FOREIGN KEY (Doctor_ID) REFERENCES Doctor(Doctor_ID)
    -> );
Query OK, 0 rows affected (0.322 sec)

mysql> INSERT INTO Patient VALUES
    -> (1, 'Arun', 25, 'Chennai'),
    -> (2, 'Priya', 30, 'Madurai'),
    -> (3, 'Rahul', 40, 'Coimbatore'),
    -> (4, 'Divya', 22, 'Chennai');
Query OK, 4 rows affected (0.055 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql>
mysql> INSERT INTO Doctor VALUES
    -> (101, 'Dr. Kumar', 'Cardiology'),
    -> (102, 'Dr. Meena', 'Neurology'),
    -> (103, 'Dr. Ravi', 'Orthopedics');
Query OK, 3 rows affected (0.023 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Appointment VALUES
    -> (1001, 1, 101, '2026-10-20', 'Scheduled'),
    -> (1002, 2, 102, '2026-10-21', 'Completed'),
    -> (1003, 1, 102, '2026-10-22', 'Scheduled');
Query OK, 3 rows affected (0.085 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

a) Write an SQL query using `INNER JOIN` to display the patient name, doctor name, specialization, and appointment date for all scheduled appointments.

```
mysql> SELECT
    ->     p.Patient_Name,
    ->     d.Doctor_Name,
    ->     d.Specialization,
    ->     a.Appointment_Date
    -> FROM Patient p
    -> INNER JOIN Appointment a
    ->     ON p.Patient_ID = a.Patient_ID
    -> INNER JOIN Doctor d
    ->     ON a.Doctor_ID = d.Doctor_ID
    -> WHERE a.Status = 'Scheduled';
+--------------+-------------+----------------+------------------+
| Patient_Name | Doctor_Name | Specialization | Appointment_Date |
+--------------+-------------+----------------+------------------+
| Arun         | Dr. Kumar   | Cardiology     | 2026-10-20       |
| Arun         | Dr. Meena   | Neurology      | 2026-10-22       |
+--------------+-------------+----------------+------------------+
2 rows in set (0.008 sec)
```

b) Write an SQL query using `LEFT JOIN` to display all patients, including those who have never booked an appointment.

```
mysql> SELECT
    ->     p.Patient_ID,
    ->     p.Patient_Name,
    ->     p.Age,
    ->     p.City,
    ->     a.Appointment_ID,
    ->     a.Appointment_Date,
    ->     a.Status
    -> FROM Patient p
    -> LEFT JOIN Appointment a
    ->     ON p.Patient_ID = a.Patient_ID;
+------------+--------------+------+-----------+----------------+------------------+-----------+
| Patient_ID | Patient_Name | Age  | City      | Appointment_ID | Appointment_Date | Status    |
+------------+--------------+------+-----------+----------------+------------------+-----------+
|          1 | Arun         |   25 | Chennai   |           1003 | 2026-10-22       | Scheduled |
|          1 | Arun         |   25 | Chennai   |           1001 | 2026-10-20       | Scheduled |
|          2 | Priya        |   30 | Madurai   |           1002 | 2026-10-21       | Completed |
|          3 | Rahul        |   40 | Coimbatore |          NULL | NULL             | NULL      |
|          4 | Divya        |   22 | Chennai   |           NULL | NULL             | NULL      |
+------------+--------------+------+-----------+----------------+------------------+-----------+
5 rows in set (0.007 sec)
```

c) Write an SQL query using joins and an aggregate function to display each doctor's name and the total number of appointments assigned to that doctor, including doctors with zero appointments

```
mysql> SELECT
    ->     d.Doctor_Name,
    ->     COUNT(a.Appointment_ID) AS Total_Appointments
    -> FROM Doctor d
    -> LEFT JOIN Appointment a
    ->     ON d.Doctor_ID = a.Doctor_ID
    -> GROUP BY d.Doctor_ID, d.Doctor_Name;
+-------------+--------------------+
| Doctor_Name | Total_Appointments |
+-------------+--------------------+
| Dr. Kumar   |                  1 |
| Dr. Meena   |                  2 |
| Dr. Ravi    |                  0 |
+-------------+--------------------+
3 rows in set (0.007 sec)

mysql>
```

Question 2: Inventory Management — SQL Triggers

```
mysql> USE RetailStore;
Database changed
mysql>
mysql> CREATE TABLE Product (
    ->     Product_ID INT PRIMARY KEY,
    ->     Product_Name VARCHAR(100),
    ->     Stock_Quantity INT NOT NULL,
    ->     Price DECIMAL(10,2) NOT NULL
    -> );
Query OK, 0 rows affected (0.138 sec)

mysql>
mysql> CREATE TABLE Sales (
    ->     Sale_ID INT PRIMARY KEY,
    ->     Sale_Date DATE
    -> );
Query OK, 0 rows affected (0.128 sec)

mysql>
mysql> CREATE TABLE Sale_Details (
    ->     Sale_ID INT,
    ->     Product_ID INT,
    ->     Quantity INT NOT NULL,
    ->     PRIMARY KEY (Sale_ID, Product_ID),
    ->     FOREIGN KEY (Sale_ID) REFERENCES Sales(Sale_ID),
    ->     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
    -> );
Query OK, 0 rows affected (0.317 sec)

mysql> DELIMITER //
```

a) Write a `BEFORE INSERT` trigger on `Sale_Details` that prevents a sale when the requested quantity exceeds the available stock. It should also reject zero or negative quantities.

```
mysql> CREATE TRIGGER Before_Sale_Details_Insert
    -> BEFORE INSERT ON Sale_Details
    -> FOR EACH ROW
    -> BEGIN
    ->     DECLARE v_stock INT DEFAULT NULL;
    ->
    ->     IF NEW.Quantity <= 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Quantity must be greater than zero';
    ->     END IF;
    ->
    ->     SELECT Stock_Quantity
    ->     INTO v_stock
    ->     FROM Product
    ->     WHERE Product_ID = NEW.Product_ID
    ->     FOR UPDATE;
    ->
    ->     IF v_stock IS NULL THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Product does not exist';
    ->     END IF;
    ->
    ->     IF NEW.Quantity > v_stock THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Insufficient stock';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.081 sec)

mysql>
mysql> DELIMITER ;
mysql> DELIMITER //
mysql>
```

b) Write an `AFTER INSERT` trigger on `Sale_Details` to reduce the corresponding product's stock after a valid sale.

```
mysql>
mysql> DELIMITER ;
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER After_Sale_Details_Insert
    -> AFTER INSERT ON Sale_Details
    -> FOR EACH ROW
    -> BEGIN
    ->     UPDATE Product
    ->     SET Stock_Quantity = Stock_Quantity - NEW.Quantity
    ->     WHERE Product_ID = NEW.Product_ID;
    -> END //
Query OK, 0 rows affected (0.077 sec)

mysql>
mysql> DELIMITER ;
```

c) Demonstrate the triggers using suitable sample records and SQL statements. Show what happens when the requested quantity is greater than the available stock

```
mysql> DELIMITER ;
mysql> INSERT INTO Product VALUES
    -> (1, 'Laptop', 10, 50000.00),
    -> (2, 'Mouse', 20, 500.00),
    -> (3, 'Keyboard', 15, 1200.00);
Query OK, 3 rows affected (0.070 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Sales VALUES
    -> (101, '2026-10-16'),
    -> (102, '2026-10-16'),
    -> (103, '2026-10-16');
Query OK, 3 rows affected (0.045 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Sale_Details
    -> VALUES (101, 1, 2);
Query OK, 1 row affected (0.042 sec)

mysql>
mysql> INSERT INTO Sale_Details
    -> VALUES (101, 2, 5);
Query OK, 1 row affected (0.026 sec)

mysql>
mysql> SELECT * FROM Product;
+------------+--------------+----------------+----------+
| Product_ID | Product_Name | Stock_Quantity | Price    |
+------------+--------------+----------------+----------+
|          1 | Laptop       |              8 | 50000.00 |
|          2 | Mouse        |             15 |   500.00 |
|          3 | Keyboard     |             15 |  1200.00 |
+------------+--------------+----------------+----------+
3 rows in set (0.002 sec)

mysql> SELECT * FROM Sale_Details;
+---------+------------+----------+
| Sale_ID | Product_ID | Quantity |
+---------+------------+----------+
```

```
| Sale_ID | Product_ID | Quantity |
+---------+------------+----------+
|     101 |          1 |        2 |
|     101 |          2 |        5 |
+---------+------------+----------+
2 rows in set (0.002 sec)

mysql>
mysql> INSERT INTO Sale_Details
    -> VALUES (102, 1, 20);
ERROR 1644 (45000): Insufficient stock
mysql>
mysql> INSERT INTO Sale_Details
    -> VALUES (102, 2, 0);
ERROR 1644 (45000): Quantity must be greater than zero
mysql>
mysql> INSERT INTO Sale_Details
    -> VALUES (103, 3, 3);
Query OK, 1 row affected (0.029 sec)

mysql>
mysql> SELECT * FROM Product;
+------------+--------------+----------------+----------+
| Product_ID | Product_Name | Stock_Quantity | Price    |
+------------+--------------+----------------+----------+
|          1 | Laptop       |              8 | 50000.00 |
|          2 | Mouse        |             15 |   500.00 |
|          3 | Keyboard     |             12 |  1200.00 |
+------------+--------------+----------------+----------+
3 rows in set (0.001 sec)

mysql> SELECT * FROM Sale_Details;
+---------+------------+----------+
| Sale_ID | Product_ID | Quantity |
+---------+------------+----------+
|     101 |          1 |        2 |
|     101 |          2 |        5 |
|     103 |          3 |        3 |
+---------+------------+----------+
3 rows in set (0.004 sec)
```

Question 3: Library Management — Stored Procedures

```
mysql> CREATE DATABASE liblab;
Query OK, 1 row affected (0.111 sec)

mysql> USE liblab;
Database changed
mysql> CREATE TABLE Book (
    ->     Book_ID INT PRIMARY KEY,
    ->     Book_Title VARCHAR(100),
    ->     Available_Copies INT NOT NULL
    -> );
Query OK, 0 rows affected (0.173 sec)

mysql>
mysql> CREATE TABLE Member (
    ->     Member_ID INT PRIMARY KEY,
    ->     Member_Name VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.118 sec)

mysql>
mysql> CREATE TABLE Book_Issue (
    ->     Issue_ID INT PRIMARY KEY AUTO_INCREMENT,
    ->     Member_ID INT,
    ->     Book_ID INT,
    ->     Issue_Date DATE,
    ->     Return_Date DATE DEFAULT NULL,
    ->     FOREIGN KEY (Member_ID) REFERENCES Member(Member_ID),
    ->     FOREIGN KEY (Book_ID) REFERENCES Book(Book_ID)
    -> );
Query OK, 0 rows affected (0.323 sec)

mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE IssueBook(
    ->     IN p_Member_ID INT,
    ->     IN p_Book_ID INT
    -> )
    -> BEGIN
    ->     DECLARE v_member_count INT DEFAULT 0;
    ->     DECLARE v_available_copies INT DEFAULT NULL;
```

a) Write a stored procedure named `IssueBook` that accepts a member ID and a book ID as input parameters. It should check whether the member and book exist and whether copies are available. If valid, it should create an issue record and reduce the available copies by one.

```
-> BEGIN
->     DECLARE v_member_count INT DEFAULT 0;
->     DECLARE v_available_copies INT DEFAULT NULL;
->
->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
->     BEGIN
->         ROLLBACK;
->         RESIGNAL;
->     END;
->
->     START TRANSACTION;
->
->     SELECT COUNT(*)
->     INTO v_member_count
->     FROM Member
->     WHERE Member_ID = p_Member_ID;
->
->     IF v_member_count = 0 THEN
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT = 'Member does not exist';
->     END IF;
->
->     SELECT Available_Copies
->     INTO v_available_copies
->     FROM Book
->     WHERE Book_ID = p_Book_ID
->     FOR UPDATE;
->
->     IF v_available_copies IS NULL THEN
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT = 'Book does not exist';
->     END IF;
->
->     IF v_available_copies <= 0 THEN
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT = 'No copies available';
->     END IF;
->
->     INSERT INTO Book_Issue
```

```
    ->     INSERT INTO Book_Issue
    ->         (Member_ID, Book_ID, Issue_Date, Return_Date)
    ->     VALUES
    ->         (p_Member_ID, p_Book_ID, CURDATE(), NULL);
    ->
    ->     UPDATE Book
    ->     SET Available_Copies = Available_Copies - 1
    ->     WHERE Book_ID = p_Book_ID;
    ->
    ->     COMMIT;
    -> END //
Query OK, 0 rows affected (0.067 sec)

mysql>
mysql> DELIMITER ;
```

b) Write a stored procedure named `ReturnBook` that accepts an issue ID as an input parameter. It should record the return date and increase the book's available copies by one. It must prevent a book from being returned more than once.

```
mysql> CREATE PROCEDURE ReturnBook(
    ->     IN p_Issue_ID INT
    -> )
    -> BEGIN
    ->     DECLARE v_book_id INT DEFAULT NULL;
    ->     DECLARE v_return_date DATE DEFAULT NULL;
    ->     DECLARE v_issue_count INT DEFAULT 0;
    ->
    ->     DECLARE EXIT HANDLER FOR SQLEXCEPTION
    ->     BEGIN
    ->         ROLLBACK;
    ->         RESIGNAL;
    ->     END;
    ->
    ->     START TRANSACTION;
    ->
    ->     SELECT COUNT(*)
    ->     INTO v_issue_count
    ->     FROM Book_Issue
    ->     WHERE Issue_ID = p_Issue_ID;
    ->
    ->     IF v_issue_count = 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Issue record does not exist';
    ->     END IF;
    ->
    ->     SELECT Book_ID, Return_Date
    ->     INTO v_book_id, v_return_date
    ->     FROM Book_Issue
    ->     WHERE Issue_ID = p_Issue_ID
    ->     FOR UPDATE;
    ->
    ->     IF v_return_date IS NOT NULL THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Book has already been returned';
    ->     END IF;
    ->
    ->     UPDATE Book_Issue
    ->     SET Return_Date = CURDATE()
    ->     WHERE Issue_ID = p_Issue_ID;
    ->
```

```
    ->
    ->     UPDATE Book
    ->     SET Available_Copies = Available_Copies + 1
    ->     WHERE Book_ID = v_book_id;
    ->
    ->     COMMIT;
    -> END //
Query OK, 0 rows affected (0.075 sec)

mysql>
mysql> DELIMITER ;
```

c) Execute both procedures using suitable sample data and display the updated book availability and issue records.

```
mysql> INSERT INTO Book (Book_ID, Book_Title, Available_Copies)
    -> VALUES
    -> (1, 'Database Systems', 5),
    -> (2, 'Computer Networks', 3),
    -> (3, 'Operating Systems', 2);
Query OK, 3 rows affected (0.069 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Member (Member_ID, Member_Name)
    -> VALUES
    -> (101, 'Arun'),
    -> (102, 'Priya'),
    -> (103, 'Rahul');
Query OK, 3 rows affected (0.024 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> CALL IssueBook(101, 1);
Query OK, 0 rows affected (0.032 sec)

mysql> CALL IssueBook(102, 2);
Query OK, 0 rows affected (0.020 sec)

mysql> CALL IssueBook(103, 1);
Query OK, 0 rows affected (0.019 sec)

mysql>
mysql> SELECT * FROM Book;
+---------+------------------+------------------+
| Book_ID | Book_Title       | Available_Copies |
+---------+------------------+------------------+
|       1 | Database Systems |                3 |
|       2 | Computer Networks |               2 |
|       3 | Operating Systems |               2 |
+---------+------------------+------------------+
3 rows in set (0.002 sec)

mysql> SELECT * FROM Book_Issue;
+----------+-----------+---------+------------+-------------+
| Issue_ID | Member_ID | Book_ID | Issue_Date | Return_Date |
```

```
mysql> SELECT * FROM Book_Issue;
+----------+-----------+---------+------------+-------------+
| Issue_ID | Member_ID | Book_ID | Issue_Date | Return_Date |
+----------+-----------+---------+------------+-------------+
|        1 |       101 |       1 | 2026-10-16 | NULL        |
|        2 |       102 |       2 | 2026-10-16 | NULL        |
|        3 |       103 |       1 | 2026-10-16 | NULL        |
+----------+-----------+---------+------------+-------------+
3 rows in set (0.001 sec)

mysql>
mysql> CALL ReturnBook(1);
Query OK, 0 rows affected (0.026 sec)

mysql>
mysql> SELECT * FROM Book;
+---------+------------------+-----------------+
| Book_ID | Book_Title       | Available_Copies |
+---------+------------------+-----------------+
|       1 | Database Systems |               4 |
|       2 | Computer Networks |               2 |
|       3 | Operating Systems |               2 |
+---------+------------------+-----------------+
3 rows in set (0.001 sec)

mysql> SELECT * FROM Book_Issue;
+----------+-----------+---------+------------+-------------+
| Issue_ID | Member_ID | Book_ID | Issue_Date | Return_Date |
+----------+-----------+---------+------------+-------------+
|        1 |       101 |       1 | 2026-10-16 | 2026-10-16  |
|        2 |       102 |       2 | 2026-10-16 | NULL        |
|        3 |       103 |       1 | 2026-10-16 | NULL        |
+----------+-----------+---------+------------+-------------+
3 rows in set (0.004 sec)

mysql>
```

GITHUB LINK:
https://github.com/AdithiMK/dbms